

Instructions

- Assignments must be written in hand.
- Write the programming in Python.
- Submit before or on the final exam day.

1. Write a program that asks for a user's birth year and calculates their current age.

```
from datetime import datetime
birth_year = int(input("Enter your birth year: "))
current_year = datetime.now().year
age = current_year - birth_year
print("Your current age is:", age)
```

2. Write a program that takes a year as input and determines if it is a leap year.

```
year = int(input("Enter a year: "))
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print(year, "is a leap year.")
else:
    print(year, "is not a leap year.")
```

3. Write a program that prints the first 10 numbers of the Fibonacci sequence using a loop.

```
n = 10
a = 0
b = 1
print("Fibonacci sequence:")
for i in range(n):
    print(a, end=" ")
    temp = a + b
    a = b
    b = temp
```

4. Problem: Given a list of unsorted integers, find the largest number without using built-in functions.

```
def find_max(numbers):
    maximum = numbers[0]
    for num in numbers:
        if num > maximum:
            maximum = num
    return maximum
```

```
# start here
arr = [10, 45, 2, 89, 33, 100, 67]
```

```
result = find_max(arr)
print("Maximum value is:", result)
```

5. Write a function that takes a string and returns a dictionary or map showing how many times each character appears.

```
def char_count(text):
    text = text.lower()
    result = {}
    for char in text:
        if char in result:
            result[char] += 1
        else:
            result[char] = 1
    return result
```

```
# Example usage
string = input("Enter a string: ")
print(char_count(string))
```

6. Define a Student class with attributes for name, ID, and a list of grades, including a method to calculate the average grade.

```
class Student:
    def __init__(self, name, student_id, grades=None):
        self.name = name
        self.student_id = student_id
        self.grades = grades if grades is not None else []
    def add_grade(self, grade):
        self.grades.append(grade)
    def calculate_average(self):
        if not self.grades:
            return 0
        return sum(self.grades) / len(self.grades)
```

```
# Example usage
student = Student("Rahim", "S101", [80, 85, 90])
print("Name:", student.name)
print("ID:", student.student_id)
student.add_grade(95)
```

```
print("Grades:", student.grades)

print("Average:", student.calculate_average())
```

7. Implement a linear search function that finds and returns the index of a target name in a list of names.

```
def linear_search(names, target):
    for index, name in enumerate(names):
        if name == target:
            return index
    return -1
```

Example usage

```
names = ["Rahim", "Karim", "Habib", "Ayesha"]
result = linear_search(names, "Habib")
print(result) # Output: 2
```

8. Implement the selection sort algorithm to sort an array of integers in ascending order.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        # swap
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr
```

Example

```
numbers = [10, 20, 3, 40, 2, 1, 90]
sorted_numbers = selection_sort(numbers)
print(sorted_numbers)
```

9. Write a program using a stack data structure to check if a string of mathematical parentheses is perfectly balanced.

```
Def is_balanced(s):
    stack = []
    for ch in s:
        if ch == '(' or ch == '{' or ch == '[':
            stack.append(ch)
        else:
            if not stack:
                return False
            top = stack.pop()
            if (ch == ')' and top != '(') or \
               (ch == '}' and top != '{') or \
               (ch == ']' and top != '['):
                return False
    return len(stack) == 0
```

```
# Example
expr = "{[()]}"
if is_balanced(expr) == True:
    print("Balanced")
else:
    print("Imbalanced")
```

10. Write a program that simulates memory allocation by finding contiguous available blocks in a 4096 byte array and marking them as used.

```
Def allocate_memory(memory, size):
    n = len(memory)
    for i in range(n - size + 1):
        can_allocate = True
        # check continuous block

        for j in range(i, i + size):
            if memory[j] == True:
                can_allocate = False
                break

        # if found free block
        if can_allocate:
            for j in range(i, i + size):
                memory[j] = True
            return i # starting index of allocation
    return -1 # not found
```

```
# Example usage
```

```

memory = [False, True, False, False]
size = 2
index = allocate_memory(memory, size)
print("Allocated at index:", index)
print("Updated memory:", memory)

```

11. Draw the recursion tree for calculating the 5th Fibonacci number and identify the overlapping subproblems.

```

def fib(n):
    if n <= 1:
        return n

    a = 0
    b = 1

    for I in range(2, n + 1): Example: like C++: for (int I = 2; I <= n; i++)
        temp = a + b
        a = b
        b = temp

    return b

print("F(5) =", fib(5))

```

12. Draw the recursion tree showing all possible paths to make change for the amount 4 using coin denominations of 1, 2, and 3.

```

          4
        /-----|-----\
       3         2         1
     /---|---\  /---|---\  /---|---\
    2  1  0  1  0  -1  0  -1  -2
  /---|---\  |  /\
 1  0  -1  0  0  -1  -2
/\
0  -1  -2

```

```

def count_ways(n):
    if n == 0:
        return 1
    if n < 0:
        return 0

```

```
return count_ways(n-1) + count_ways(n-2) + count_ways(n-3)
```

```
# Example  
print(count_ways(4))
```

13. Draw the recursion tree for the possible paths a robot can take to reach the bottom-right corner of a 3x3 grid, moving only right and down.

```
def count_paths(l, j, n, m):  
    if l == n - 1 and j == m - 1:  
        return 1  
  
    if l >= n or j >= m:  
        return 0  
  
    right = count_paths(l, j + 1, n, m)  
    down = count_paths(l + 1, j, n, m)  
  
    return right + down  
print(count_paths(0, 0, 3, 3))
```

14. Trace the step-by-step execution flow and variable states for a loop that determines if the number 7 is prime.

```
num = 7  
  
isPrime = True  
  
print("Number =", num)  
  
print()  
  
print("Step\t\t\t7%\t\tisPrime")  
  
print("-----")  
  
print("Start\t\t\t\t\ttrue")  
  
step = 1  
  
for i in range(2, num):  
    mod = num % i  
  
    if mod == 0:
```

```

isPrime = False

print(step, "\t", i, "\t", mod, "\t", "true" if isPrime else "false")

step += 1

print("-----")

```

```

if isPrime:

```

```

    print("Final Output: Prime Number")

```

```

else:

```

```

    print("Final Output: Not Prime Number")

```

```

n = 7
prime = True
for i in range(2, n):
    if n % i == 0:
        prime = False
        break
if prime:
    print("Prime")
else:
    print("Not Prime")

```

i	n % i	Result
2	1	Continue
3	1	Continue
4	3	Continue
5	2	Continue
6	1	Continue

15. Track the step-by-step execution path, including low, mid, and high index values, when performing a binary search for the number 14 in the sorted array [3, 8, 14, 22, 35, 41, 56, 78].

Given Array list:

[3, 8, 14, 22, 35, 41, 56, 78]

```

def binary_search(arr, target):

```

```

    low = 0

```

```

high = len(arr) - 1
while low <= high:
    mid = (low + high)
    if arr[mid] == target:
        return True
    elif arr[mid] < target:
        low = mid + 1
    else:
        high = mid - 1
return False

# Only pass the array here
arr = [3, 8, 14, 22, 35, 41, 56, 78]
target = 14
if binary_search(arr, target):
    print("FOUND ✔")
else:
    print("NOT FOUND ✖")

```

Iter	Low	High	Mid	Value at Mid	Logic Check	Next Action
1	0	7	3	22	14 < 22	high = mid - 1 = 2
2	0	2	1	8	14 > 8	low = mid + 1 = 2
3	2	2	2	14	14 == 14	FOUND ✔